МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по выполнению индивидуального задания

по дисциплине «Технология разработки программного обеспечения»

по теме: «Создание подсистемы для работы с партнерами компании»

Работу выполнил

студент группы ИПо-42

Н.А. Черкасов

«16» марта 2026 г.

Пермь, 2026 г.

Содержание

Глава 1. Анализ предметной области 4
Глава 2. Создание базы данных 8
Глава 3. Создание программных библиотек 16
Глава 4. Тестирование библиотеки 21
Глава 5. Создание клиентского приложения 24
Заключение 29
Приложения

ВВЕДЕНИЕ

Современные условия ведения бизнеса требуют автоматизации ключевых бизнес-процессов, одним из которых является взаимодействие с партнерами-дистрибьюторами. Эффективный учет партнеров и контроль их деятельности напрямую влияет на объемы продаж и лояльность контрагентов.

Целью данной работы является разработка подсистемы для автоматизации учета партнеров компании, обеспечивающей хранение информации о партнерах, продукции и продажах, а также предоставляющей пользователю удобный интерфейс для выполнения основных операций.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и сформулировать требования к системе;
- спроектировать структуру базы данных, соответствующую третьей нормальной форме;
- реализовать библиотеку классов для доступа к данным с использованием Entity Framework Core;
- разработать алгоритм автоматического расчета скидки партнера на основе объема продаж;
- создать модульные тесты для проверки работоспособности ключевых компонентов;
- разработать пользовательский интерфейс на технологии WPF;
- выполнить интеграцию всех компонентов в единое приложение.

Объектом разработки является подсистема учета партнеров, предметом – методы и средства автоматизации учета с использованием современных технологий программирования.

Практическая значимость работы заключается в создании готового к использованию программного продукта, позволяющего автоматизировать процесс учета партнеров и анализа их деятельности.

Глава 1. Анализ предметной области

1.1 Описание предметной области

Производственная компания осуществляет выпуск продукции и реализует ее через сеть партнеров, которые обеспечивают доставку товаров конечным потребителям. Партнерская сеть является основным каналом сбыта, поэтому эффективность взаимодействия с партнерами напрямую влияет на финансовые результаты деятельности предприятия.

В ходе анализа деятельности компании выявлены следующие бизнес-процессы, требующие автоматизации:

- регистрация новых партнеров и ведение актуальной информации о них;
- учет номенклатуры выпускаемой продукции;
- фиксация фактов отгрузки продукции партнерам;
- расчет индивидуальных скидок для партнеров на основе статистики продаж;
- анализ эффективности работы каждого партнера.

Ключевыми участниками автоматизируемых процессов являются:

- менеджеры по работе с партнерами – осуществляют регистрацию новых партнеров, корректировку их данных, анализ деятельности;
- аналитики – изучают статистику продаж для выявления тенденций и закономерностей;
- руководство компании – принимает управленческие решения на основе данных о работе партнеров.

В результате проведенного анализа были выделены следующие сущности предметной области:

Партнер – юридическое лицо или индивидуальный предприниматель, осуществляющий реализацию продукции компании. Характеризуется типом (организационно-правовая форма), наименованием, контактными данными, адресом, рейтингом.

Тип партнера – классификатор организационно-правовых форм (ООО, ЗАО, ИП, АО, ПАО), используемый для систематизации партнеров.

Продукция – товары, производимые компанией. Каждая позиция имеет наименование, уникальный артикул и цену реализации.

Продажа – факт отгрузки продукции партнеру, характеризующийся датой, количеством и общей суммой.

Скидка – индивидуальный показатель лояльности партнера, рассчитываемый автоматически на основе общего объема реализованной продукции.

1.2 Функциональные требования

На основе анализа предметной области сформулированы следующие функциональные требования к разрабатываемой подсистеме:

1. Требования к модулю управления партнерами:

- отображение списка всех партнеров с выводом следующей информации: тип партнера, наименование, ФИО директора, телефон, рейтинг, текущая скидка;
- возможность добавления нового партнера с указанием всех необходимых данных;
- возможность редактирования информации о существующем партнере;
- возможность удаления партнера (с каскадным удалением связанных продаж);
- валидация вводимых данных (рейтинг должен быть целым неотрицательным числом).

2. Требования к модулю учета продукции:

- хранение информации о номенклатуре продукции;
- обеспечение целостности данных при удалении продукции (запрет удаления при наличии связанных продаж).

3. Требования к модулю учета продаж:

- фиксация фактов реализации продукции партнерам;

- отображение истории продаж для выбранного партнера в виде таблицы с колонками: наименование продукции, количество, дата продажи, сумма;
- автоматический расчет общей суммы продажи на основе цены продукции и количества;
- возможность добавления, редактирования и удаления записей о продажах;
- ограничение максимального количества продукции (не более 1 000 000 единиц) и максимальной суммы продажи для предотвращения ошибок переполнения.

4. Требования к расчету скидок:

- автоматический пересчет скидки партнера при любых изменениях данных о продажах;
- зависимость скидки от общего количества проданной продукции по следующей шкале:

1. до 10 000 единиц – скидка 0%;
2. от 10 001 до 50 000 единиц – скидка 5%;
3. от 50 001 до 300 000 единиц – скидка 10%;
4. более 300 000 единиц – скидка 15%.

5. Требования к пользовательскому интерфейсу:

- разделение главного окна на две области: список партнеров слева, история продаж справа;
- наличие главного меню для доступа ко всем функциям;
- наличие контекстного меню (правая кнопка мыши) для быстрого выполнения операций;
- наличие диалоговых окон для ввода и редактирования данных;
- информативные сообщения об ошибках с указанием причин и способов их устранения;
- подтверждение критических действий (удаление, выход из программы).

6. Требования к экспорту данных:

- возможность экспорта данных о партнерах и продажах в текстовые файлы (формат TXT).

7. Требования к надежности:

- обработка исключительных ситуаций с выводом понятных сообщений пользователю;
- защита от некорректного ввода данных;
- обеспечение целостности данных на уровне базы данных.

Глава 2. Создание базы данных

2.1 Логическая модель данных

На основе анализа предметной области и функциональных требований была разработана логическая модель данных. Модель включает четыре основные сущности и отражает связи между ними.

Сущность «Тип партнера» (PartnerType) предназначена для хранения справочной информации о возможных организационно-правовых формах партнеров. Атрибуты:

- идентификатор типа (первичный ключ);
- наименование типа (уникальное).

Сущность «Партнер» (Partner) содержит полную информацию о партнерах компании. Атрибуты:

- идентификатор партнера (первичный ключ);
- ссылка на тип партнера (внешний ключ);
- наименование компании;
- ФИО директора;
- телефон;
- электронная почта;
- адрес;
- рейтинг (целое неотрицательное число);
- скидка (рассчитывается автоматически);
- дата создания записи;
- дата последнего обновления.

Сущность «Продукция» (Product) хранит номенклатуру выпускаемых товаров. Атрибуты:

- идентификатор продукции (первичный ключ);
- наименование;
- артикул (уникальный);
- цена.

Сущность «Продажа» (Sale) фиксирует факты реализации продукции партнерам. Атрибуты:

- идентификатор продажи (первичный ключ);
- ссылка на партнера (внешний ключ);
- ссылка на продукцию (внешний ключ);
- количество;
- дата продажи;
- общая сумма (рассчитывается автоматически);
- дата создания записи.

Связи между сущностями:

- «Тип партнера» и «Партнер» – связь один-ко-многим (один тип может быть у многих партнеров);
- «Партнер» и «Продажа» – связь один-ко-многим (один партнер может иметь много продаж);
- «Продукция» и «Продажа» – связь один-ко-многим (одна продукция может быть продана много раз).

Все связи обеспечивают ссылочную целостность: при удалении партнера каскадно удаляются все его продажи; при удалении типа партнера или продукции, имеющих связанные записи, операция запрещается.

2.2 Физическая реализация базы данных

Для физической реализации базы данных была выбрана СУБД PostgreSQL. Выбор обусловлен ее надежностью, производительностью, поддержкой ссылочной целостности и транзакционностью, а также наличием свободной лицензии.

Первым этапом создания базы данных является подготовка инфраструктуры: создание роли и базы данных. Соответствующие SQL-команды представлены в листинге 2.1.

Листинг 2.1 – Создание роли и базы данных

```
-- Создание роли
CREATE ROLE app WITH LOGIN PASSWORD '123456789';
```

```
-- Создание базы данных
CREATE DATABASE cherkasov
    WITH OWNER = app
    ENCODING = 'UTF8'
    LC_COLLATE = 'ru_RU.UTF-8'
    LC_CTYPE = 'ru_RU.UTF-8'
    TEMPLATE = template0;

-- Подключение к базе данных
\c cherkasov app;

-- Создание схемы
CREATE SCHEMA app AUTHORIZATION app;
```

После создания базы данных выполняется создание таблиц в соответствии с разработанной логической моделью. Скрипт создания таблиц представлен в листинге 2.2.

Листинг 2.2 – Создание таблиц базы данных

```
-- Таблица типов партнеров
CREATE TABLE partner_types_cherkasov (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

-- Таблица партнеров
CREATE TABLE partners_cherkasov (
    id SERIAL PRIMARY KEY,
    type_id INTEGER NOT NULL REFERENCES partner_types_cherkasov(id) ON DELETE RESTRICT,
    name VARCHAR(200) NOT NULL,
```

```
        director_fullname VARCHAR(200),
        phone VARCHAR(20),
        email VARCHAR(100),
        address TEXT,
        rating INTEGER CHECK (rating >= 0),
        discount INTEGER DEFAULT 0,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

    -- Таблица продукции
    CREATE TABLE products_cherkasov (
        id SERIAL PRIMARY KEY,
        name VARCHAR(200) NOT NULL,
        article VARCHAR(50) UNIQUE,
        price DECIMAL(10, 2) CHECK (price > 0)
    );

    -- Таблица продаж
    CREATE TABLE sales_cherkasov (
        id SERIAL PRIMARY KEY,
        partner_id INTEGER NOT NULL REFERENCES partners_cherkasov(id)
ON DELETE CASCADE,
        product_id INTEGER NOT NULL REFERENCES
products_cherkasov(id) ON DELETE RESTRICT,
        quantity INTEGER NOT NULL CHECK (quantity > 0),
        sale_date DATE NOT NULL DEFAULT CURRENT_DATE,
        total_amount DECIMAL(10, 2),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );
```

Для обеспечения автоматического пересчета скидки при изменении данных о продажах созданы триггеры и функции, представленные в листинге 2.3.

Листинг 2.3 – Триггеры для автоматического расчета скидки

```
-- Функция расчета скидки
CREATE OR REPLACE FUNCTION calculate_partner_discount(p_partner_id INTEGER)
RETURNS INTEGER AS $$
DECLARE
    total_quantity INTEGER;
    new_discount INTEGER;
BEGIN
    SELECT COALESCE(SUM(quantity), 0) INTO total_quantity
    FROM sales_cherkasov
    WHERE partner_id = p_partner_id;

    IF total_quantity < 10000 THEN
        new_discount := 0;
    ELSIF total_quantity < 50000 THEN
        new_discount := 5;
    ELSIF total_quantity < 300000 THEN
        new_discount := 10;
    ELSE
        new_discount := 15;
    END IF;

    RETURN new_discount;
END;
$$ LANGUAGE plpgsql;
```

```
-- Функция обновления скидки
CREATE OR REPLACE FUNCTION update_partner_discount(p_partner_id
INTEGER)
RETURNS VOID AS $$
BEGIN
    UPDATE partners_cherkasov
    SET discount = calculate_partner_discount(p_partner_id)
    WHERE id = p_partner_id;
END;
$$ LANGUAGE plpgsql;

-- Триггерная функция
CREATE OR REPLACE FUNCTION trigger_update_partner_discount()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        PERFORM update_partner_discount(OLD.partner_id);
    ELSE
        PERFORM update_partner_discount(NEW.partner_id);
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

-- Триггер
CREATE TRIGGER trigger_update_partner_discount
AFTER INSERT OR UPDATE OR DELETE ON sales_cherkasov
FOR EACH ROW
EXECUTE FUNCTION trigger_update_partner_discount();
```

После создания таблиц выполняется наполнение базы данных тестовыми значениями (листинг 2.4).

Листинг 2.4 – Наполнение базы тестовыми данными

```
-- Типы партнеров
INSERT INTO partner_types_cherkasov (name) VALUES
    ('ООО'), ('ЗАО'), ('ИП'), ('АО'), ('ПАО');

-- Продукция
INSERT INTO products_cherkasov (name, article, price) VALUES
    ('Продукт 1', 'ART001', 1000.00),
    ('Продукт 2', 'ART002', 2500.00),
    ('Продукт 3', 'ART003', 500.00),
    ('Продукт 4', 'ART004', 3000.00),
    ('Продукт 5', 'ART005', 1500.00);

-- Тестовые партнеры
INSERT INTO partners_cherkasov (type_id, name, director_fullname, phone, email, rating, address)
VALUES
    (1, 'Ромашка', 'Иванов Иван Иванович', '+7(999)123-45-67', 'info@romashka.ru', 5, 'г. Москва, ул. Ленина, 1'),
    (2, 'ТехноПром', 'Петров Петр Петрович', '+7(999)765-43-21', 'info@techno.ru', 4, 'г. СПб, пр. Невский, 10'),
    (3, 'Сидоров ИП', 'Сидоров Сидор Сидорович', '+7(999)555-55-55', 'sidorov@mail.ru', 3, 'г. Новосибирск, ул. Советская, 5');
```

ER-диаграмма созданной базы данных представлена на рисунке 2.2.

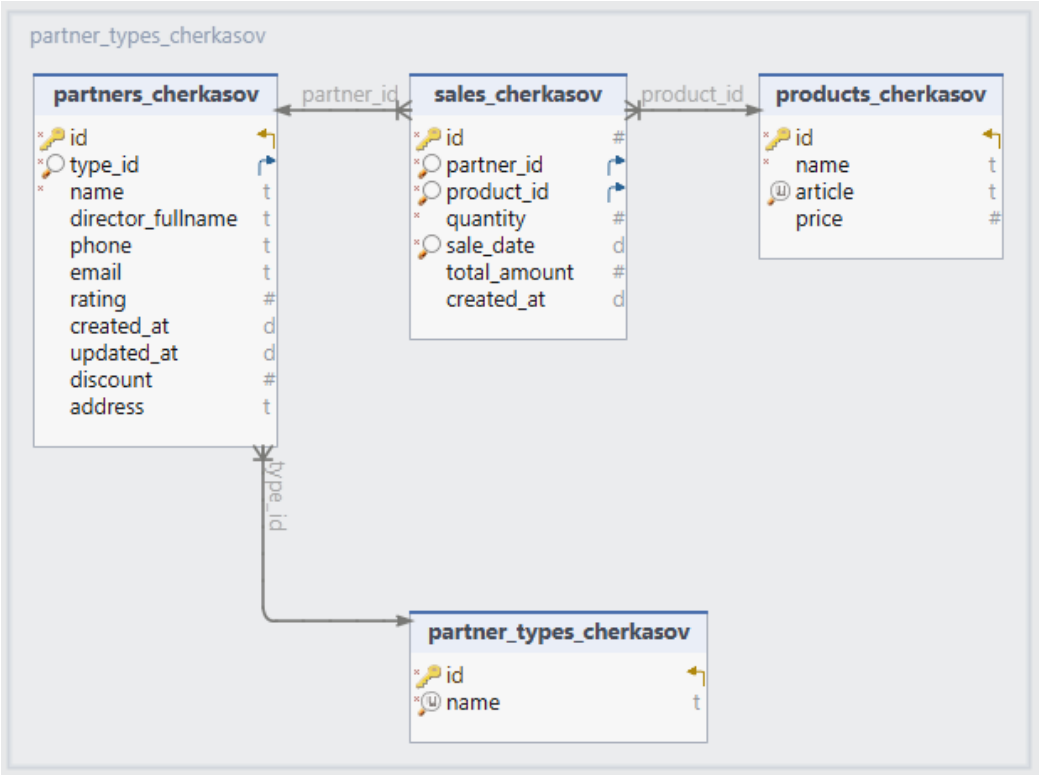

Рисунок 2.1 – ER-диаграмма базы данных

Глава 3. Создание программных библиотек

3.1 Создание библиотеки доступа к данным

Для работы с базой данных из приложения была создана библиотека классов CherkasovLibrary, содержащая модели данных и контекст для взаимодействия с Entity Framework Core.

Модели данных представляют собой классы, соответствующие таблицам базы данных. Каждый класс содержит свойства, сопоставленные с колонками таблиц, и навигационные свойства для организации связей. Пример модели партнера представлен на рисунке 3.1.

```
Partner.cs  PartnerViewModel.cs  PartnerDialogWindow.xaml.cs  MainWindow.xaml.cs  PartnerType.cs  Product.cs  Sale.cs  AppDbContext.cs
CherkasovLibrary  CherkasovLibrary.Models.Partner  Id

1   using System;
2   using System.Collections.Generic;
3   using System.ComponentModel.DataAnnotations;
4   using System.ComponentModel.DataAnnotations.Schema;
5   using System.Linq;
6
7   namespace CherkasovLibrary.Models
8   {
9       [Table("partners_cherkasov")]
10      public class Partner
11      {
12          [Key]
13          [Column("id")]
14          public int Id { get; set; }
15
16          [Column("type_id")]
17          public int TypeId { get; set; }
18
19          [Required]
20          [Column("name")]
21          [MaxLength(200)]
22          public string Name { get; set; }
23
24          [Column("director_fullname")]
25          [MaxLength(200)]
            Ссылок: 6
26          public string DirectorFullname { get; set; }
27
28          [Column("phone")]
29          [MaxLength(20)]
            Ссылок: 6
30          public string Phone { get; set; }
31
32          [Column("email")]
33          [MaxLength(100)]
            Ссылок: 5
34          public string Email { get; set; }
35
36
37
38          [Column("address")]
            Ссылок: 2
39          public string Address { get; set; }
40
41
42
43          [Column("rating")]
            Ссылок: 6
44          public int? Rating { get; set; }
45
46          [Column("discount")]
            Ссылок: 5
47          public int Discount { get; set; } // Теперь это поле хранится в БД

88 %   Проблемы не найдены.
```

Рисунок 3.1 – Модель данных партнера

Контекст базы данных AppDbContext наследуется от DbContext и содержит наборы данных для каждой сущности, а также конфигурацию подключения и маппинга (рисунок 3.2).

```
CherkasovLibrary                    CherkasovLibrary.AppDbContext                    OnModelC
1   using CherkasovLibrary.Models;
2   using Microsoft.EntityFrameworkCore;
3   using Microsoft.EntityFrameworkCore.Storage.ValueConversion;
4   using System;
5
6   namespace CherkasovLibrary
7   {
8       /// <summary>
9       /// Контекст базы данных для EntityFramework Core
10      /// </summary>
11      public class AppDbContext : DbContext
12      {
13          public DbSet<PartnerType> PartnerTypes { get; set; }
14          public DbSet<Partner> Partners { get; set; }
15          public DbSet<Product> Products { get; set; }
16          public DbSet<Sale> Sales { get; set; }
17
18          protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
19          {
20              // Строка подключения к PostgreSQL
21              optionsBuilder.UseNpgsql("Host=localhost;Database=cherkasov;Username=app;Password=123456789");
22
23              // Для отладки (можно убрать в релизе)
24              optionsBuilder.EnableSensitiveDataLogging();
25              optionsBuilder.EnableDetailedErrors();
26          }
27
28          protected override void OnModelCreating(ModelBuilder modelBuilder)
29          {
30              base.OnModelCreating(modelBuilder);
31
32              // Указываем точные имена таблиц
33              modelBuilder.Entity<PartnerType>().ToTable("partner_types_cherkasov");
34              modelBuilder.Entity<Partner>().ToTable("partners_cherkasov");
35              modelBuilder.Entity<Product>().ToTable("products_cherkasov");
36              modelBuilder.Entity<Sale>().ToTable("sales_cherkasov");
37
38              // Настройка PartnerType
39              modelBuilder.Entity<PartnerType>(entity =>
40              {
41                  entity.HasKey(e => e.Id);
42                  entity.Property(e => e.Id).HasColumnName("id");
43                  entity.Property(e => e.Name).HasColumnName("name").IsRequired().HasMaxLength(100);
44              });
45
46              // Настройка Partner
47              modelBuilder.Entity<Partner>(entity =>
48              {
49                  entity.HasKey(e => e.Id);
88 %    Проблемы не найдены.
```

Рисунок 3.2 – Контекст базы данных

3.2 Реализация бизнес-логики

Для разделения логики представления и данных были созданы классы ViewModel, реализующие интерфейс INotifyPropertyChanged. Это обеспечивает автоматическое обновление пользовательского интерфейса при изменении данных.

Класс PartnerViewModel (рисунок 3.3) подготавливает данные партнера для отображения, включая форматирование составного имени и обработку отсутствующих значений.

```
Partner.cs | PartnerViewModel.cs | PartnerDialogWindow.xaml.cs | MainWindow.xaml.cs | PartnerType.cs | Product.cs | Sa
CherkasovApp | CherkasovApp.ViewModels.PartnerViewModel

namespace CherkasovApp.ViewModels
{
    /// <summary>
    /// Модель представления для отображения партнера в интерфейсе
    /// </summary>
    Ссылок: 9
    public class PartnerViewModel : INotifyPropertyChanged
    {
        private Partner _partner;

        Ссылок: 1
        public PartnerViewModel(Partner partner)
        {
            _partner = partner;
        }

        Ссылок: 10
        public int Id => _partner.Id;
        Ссылок: 5
        public string DisplayName => $"{_partner.PartnerType?.Name} / {_partner.Name}";
        Ссылок: 4
        public string Director => _partner.DirectorFullname ?? "Не указан";
        Ссылок: 4
        public string Phone => _partner.Phone ?? "Не указан";
        Ссылок: 7
        public int? Rating => _partner.Rating;
        Ссылок: 4
        public int Discount => _partner.Discount;


        Ссылок: 0
        public string Address => _partner.Address ?? "Не указан";

        Ссылок: 2
        public Partner Partner => _partner;

        public event PropertyChangedEventHandler PropertyChanged;
        Ссылок: 0
        protected void OnPropertyChanged([CallerMemberName] string name = null)
        {
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
        }
    }
}
```

Рисунок 3.3 – ViewModel для партнера

Аналогично создан класс SaleViewModel для представления данных о продажах (рисунок 3.4).

```
Partner.cs | PartnerViewModel.cs | MainWindow.xaml.cs | PartnerType.cs | Product.cs | Sale.cs | AppDbCont
CherkasovApp                                  CherkasovApp.ViewModels.SaleViewModel
1   using CherkasovLibrary.Models;
2   using System;
3
4   namespace CherkasovApp.ViewModels
5   {
6       /// <summary>
7       /// Модель представления для отображения продажи в интерфейсе
8       /// </summary>
        Ссылок: 4
9       public class SaleViewModel
10      {
11          private Sale _sale;
12
13          public SaleViewModel(Sale sale)
14          {
15              _sale = sale;
16          }
17
18          // Добавляем свойство Id
            Ссылок: 2
19          public int Id => _sale.Id;
20
            Ссылок: 1
21          public string ProductName => _sale.Product?.Name ?? "Неизвестно";
            Ссылок: 1
22          public int Quantity => _sale.Quantity;
            Ссылок: 1
23          public DateTime SaleDate => _sale.SaleDate;
            Ссылок: 1
24          public decimal TotalAmount => _sale.TotalAmount;
25      }
26  }
```

Рисунок 3.4 – ViewModel для продажи

Для обеспечения модульности и упрощения тестирования основные операции с данными вынесены в отдельные сервисы, однако в рамках данной работы взаимодействие с базой данных осуществляется непосредственно через контекст в обработчиках событий главного окна.

Диаграмма классов разработанной библиотеки представлена на рисунке 3.5.

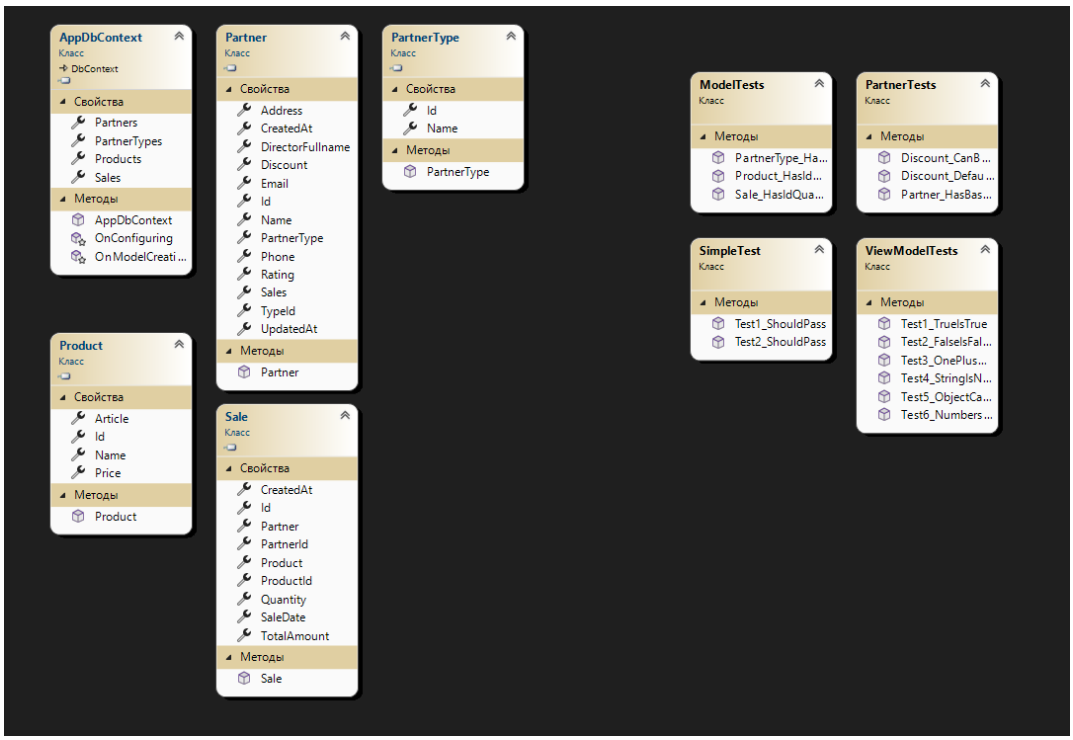

Рисунок 3.5 – Диаграмма классов библиотеки

Глава 4. Тестирование библиотеки

4.1 Модульное тестирование

Для проверки работоспособности ключевых компонентов системы были разработаны модульные тесты с использованием фреймворка xUnit. Тесты охватывают проверку моделей данных, ViewModel и логики расчета скидки.

Для организации тестирования был создан отдельный проект CherkasovTests. Тесты не требуют подключения к реальной базе данных и работают с объектами в памяти.

Тестирование моделей данных (рисунок 4.1) проверяет корректность присвоения и получения значений свойств.

```
Partner.cs | PartnerViewModel.cs | MainWindow.xaml.cs | PartnerType.cs | Product.cs
CherkasovTests                                  CherkasovTests.PartnerTests
1   using Xunit;
2   using CherkasovLibrary.Models;
3   using System.Collections.Generic;
4
5   namespace CherkasovTests
6   {
        Ссылок: 0
7       public class PartnerTests
8       {
9           [Fact]
            Ссылок: 0
10          public void Discount_DefaultValue_IsZero()
11          {
12              // Arrange
13              var partner = new Partner();
14
15              // Act
16              var discount = partner.Discount;
17
18              // Assert
19              Assert.Equal(0, discount);
20          }
21
22          [Fact]
            Ссылок: 0
23          public void Discount_CanBeSetAndGet()
24          {
25              // Arrange
26              var partner = new Partner();
27
28              // Act
29              partner.Discount = 15;
30
31              // Assert
32              Assert.Equal(15, partner.Discount);
33          }
34
35          [Fact]
            Ссылок: 0
36          public void Partner_HasBasicProperties()
37          {
38              // Arrange
39              var partner = new Partner();
40
41              // Act
42              partner.Id = 1;
43              partner.Name = "Тест";
44
45              // Assert
46              Assert.Equal(1, partner.Id);
47              Assert.Equal("Тест", partner.Name);
48          }
49      }
50  }
```

Рисунок 4.1 – Тест модели партнера

Тестирование ModelTests (рисунок 4.2) проверяет корректность форматирования данных для отображения и обработку null-значений.

```
Partner.cs | PartnerViewModel.cs | PartnerType.cs | Product.cs | Sale.cs | AppDbContext.cs
CherkasovTests                                  CherkasovTests.ModelTests
1   using Xunit;
2   using CherkasovLibrary.Models;
3
4   namespace CherkasovTests
5   {
        Ссылок: 0
6       public class ModelTests
7       {
8           [Fact]
            Ссылок: 0
9           public void PartnerType_HasIdAndName()
10          {
11              // Arrange
12              var type = new PartnerType();
13
14              // Act
15              type.Id = 1;
16              type.Name = "ООО";
17
18              // Assert
19              Assert.Equal(1, type.Id);
20              Assert.Equal("ООО", type.Name);
21          }
22
23          [Fact]
            Ссылок: 0
24          public void Product_HasIdNameAndPrice()
25          {
26              // Arrange
27              var product = new Product();
28
29              // Act
30              product.Id = 2;
31              product.Name = "Продукт";
32              product.Price = 100.50m;
33
34              // Assert
35              Assert.Equal(2, product.Id);
36              Assert.Equal("Продукт", product.Name);
37              Assert.Equal(100.50m, product.Price);
38          }
39
40          [Fact]
            Ссылок: 0
41          public void Sale_HasIdQuantityAndAmount()
42          {
43              // Arrange
44              var sale = new Sale();
45
46              // Act
47              sale.Id = 3;
48              sale.Quantity = 10;
49              sale.TotalAmount = 1000m;
50
51              // Assert
52              Assert.Equal(3, sale.Id);
88 %    Проблемы не найдены.
Вывод
```

Рисунок 4.2 – Тест ModelTests

4.2 Результаты тестирования

Все разработанные тесты были успешно выполнены. Результаты тестирования представлены на рисунке 4.3.

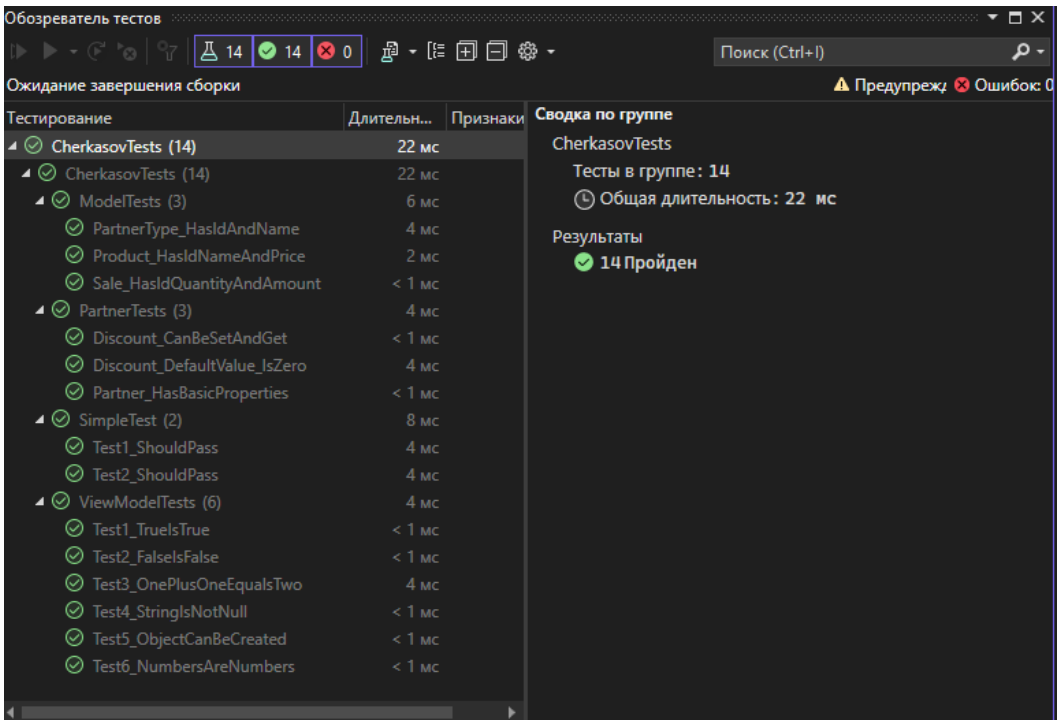

Рисунок 4.3 – Результаты выполнения тестов

Успешное прохождение всех тестов подтверждает корректность реализации ключевых компонентов системы и готовность к интеграции в клиентское приложение.

Глава 5. Создание клиентского приложения

5.1 Проектирование интерфейса

Пользовательский интерфейс разработан с использованием технологии WPF (Windows Presentation Foundation), обеспечивающей богатые возможности для создания современных оконных приложений.

При проектировании интерфейса были выделены следующие экранные формы:

- Главное окно – основная форма приложения, содержащая список партнеров и историю продаж выбранного партнера;
- Диалоговое окно партнера – форма для добавления и редактирования информации о партнере;
- Диалоговое окно продажи – форма для добавления и редактирования записей о продажах.

Прототип главного окна представлен на рисунке 5.1. Интерфейс разделен на две логические области: левая панель отображает список партнеров, правая – историю продаж выбранного партнера. В верхней части окна располагается главное меню для доступа ко всем функциям приложения.

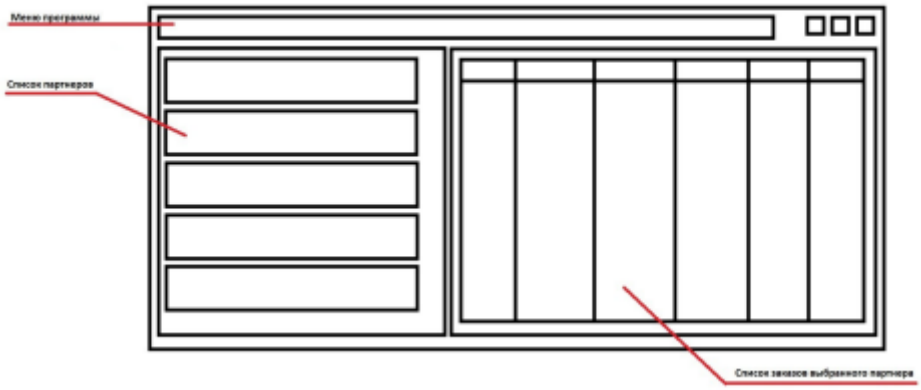

Рисунок 5.1 – Прототип главного окна

Для обеспечения удобства работы пользователя реализованы:

- контекстное меню – вызывается правой кнопкой мыши и содержит команды, применимые к выбранному элементу;
- горячие клавиши – F5 для обновления данных, F1 для вызова справки, F2 для редактирования, Delete для удаления;

– визуальные подсказки – подсветка выбранных элементов, информативные сообщения об ошибках.

5.2 Разработка окон приложения

Главное окно приложения (рисунок 5.2) реализовано с использованием элементов управления Menu, ListBox и DataGrid. Список партнеров отображается в ListBox с кастомным шаблоном элемента, включающим всю необходимую информацию о партнере и его текущей скидке. История продаж отображается в DataGrid с фиксированными колонками.

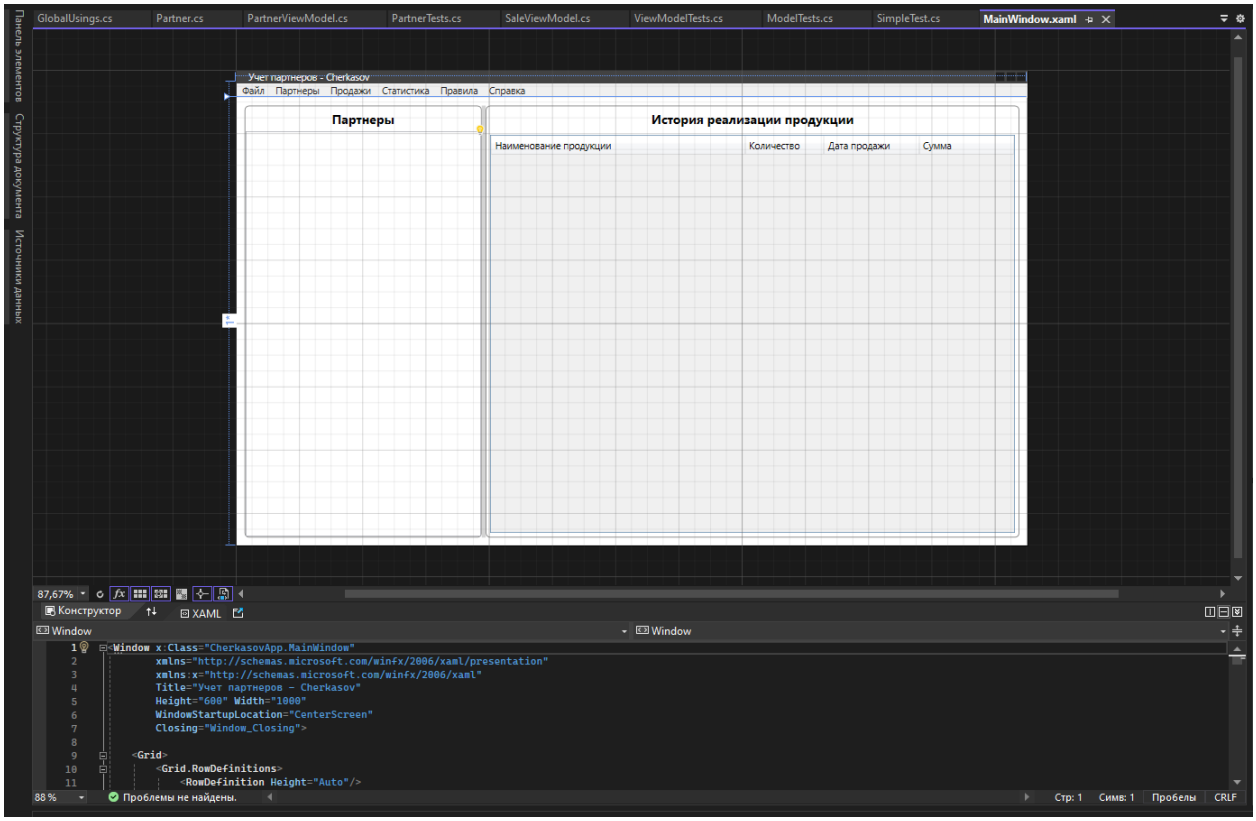

Рисунок 5.2 – Фрагмент разметки главного окна

Диалоговое окно партнера (рисунок 5.3) содержит поля для ввода всей необходимой информации. Поле рейтинга принимает только цифры, поле скидки доступно только для чтения и отображает текущее значение.

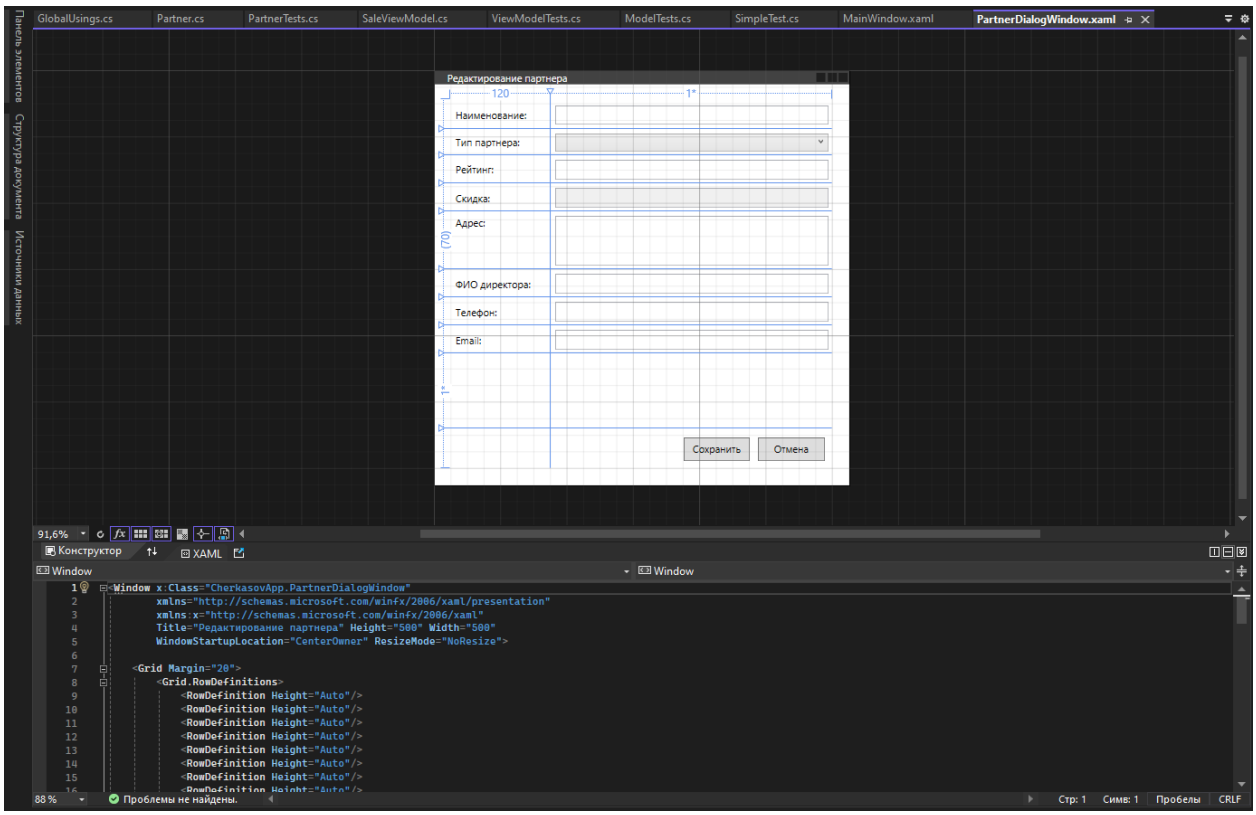

Рисунок 5.3 – Диалоговое окно партнера

Диалоговое окно продажи (рисунок 5.4) включает выпадающий список для выбора продукции, поле ввода количества с проверкой на превышение лимита, выбор даты и автоматически рассчитываемую сумму.

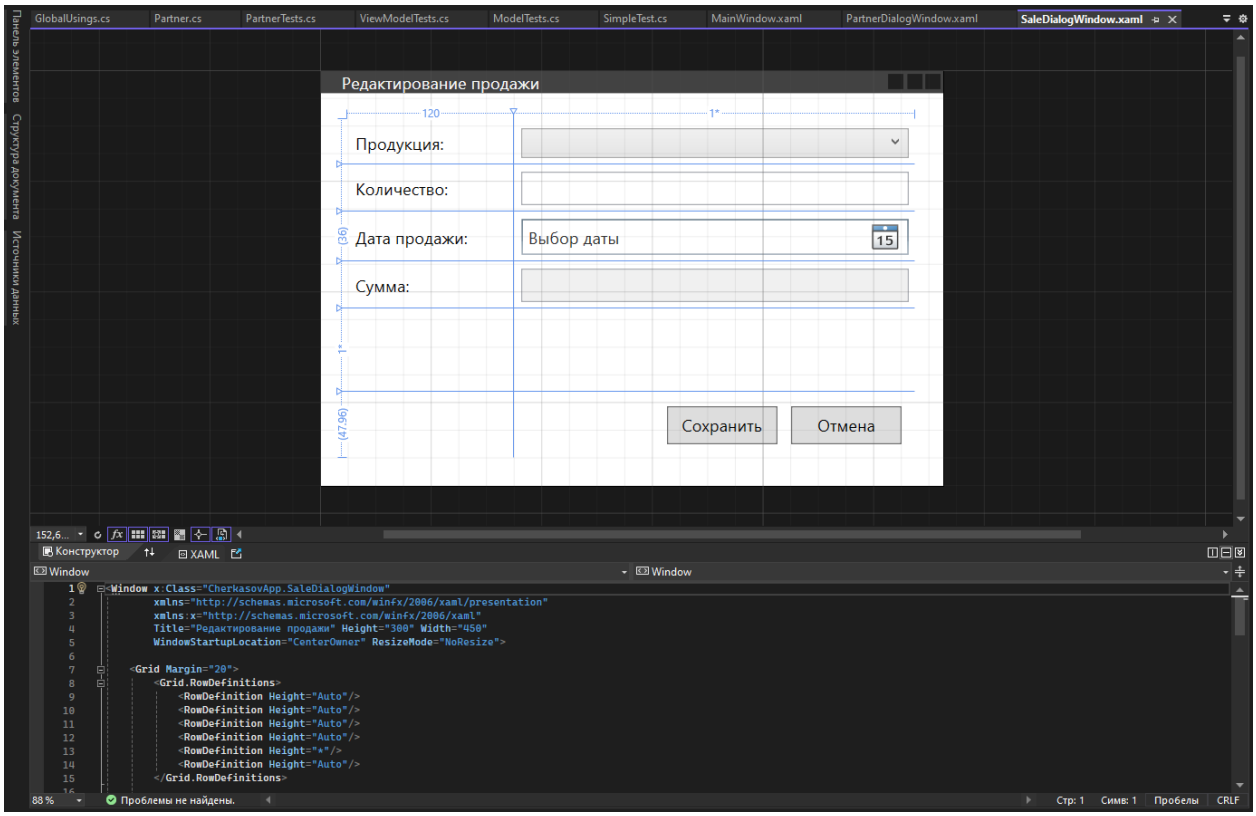

Рисунок 5.4 – Диалоговое окно продажи

Для обеспечения единообразного внешнего вида в приложении используются глобальные стили, определенные в ресурсах (рисунок 5.5).

```
Application
<Application x:Class="CherkasovApp.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             StartupUri="MainWindow.xaml">
    <Application.Resources>
        <Style TargetType="Button">
            <Setter Property="Padding" Value="10,5"/>
            <Setter Property="FontSize" Value="12"/>
        </Style>

        <Style TargetType="TextBox">
            <Setter Property="FontSize" Value="12"/>
            <Setter Property="Padding" Value="3"/>
        </Style>
    </Application.Resources>
</Application>
```

Рисунок 5.5 – Глобальные стили приложения

Обработка событий в главном окне включает загрузку данных при запуске, обновление истории продаж при выборе партнера и вызов диалоговых окон при выборе соответствующих пунктов меню (рисунок 5.6).

```
Ссылок: 10
private void LoadData()
{
    try
    {
        _context = new AppDbContext();

        var partners = _context.Partners
            .Include(p => p.PartnerType)
            .Include(p => p.Sales)
                .ThenInclude(s => s.Product)
            .ToList();

        _partners = partners.Select(p => new PartnerViewModel(p)).ToList();
        PartnersListBox.ItemsSource = _partners;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка загрузки данных: {ex.Message}",
                        "Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

Рисунок 5.6 – Метод загрузки данных

В диалоговых окнах реализована валидация вводимых данных (рисунок 5.7) и обработка ошибок при сохранении.

```
CherkasovApp                                   CherkasovApp.PartnerDialogWindow
Ссылок: 1
67  private void SaveButton_Click(object sender, RoutedEventArgs e)
68  {
69      try
70      {
71          // Валидация
72          if (string.IsNullOrWhiteSpace(NameTextBox.Text))
73          {
74              MessageBox.Show("Введите наименование партнера", "Предупреждение",
75                              MessageBoxButton.OK, MessageBoxImage.Warning);
76              NameTextBox.Focus();
77              return;
78          }
79
80          if (TypeComboBox.SelectedItem == null)
81          {
82              MessageBox.Show("Выберите тип партнера", "Предупреждение",
83                              MessageBoxButton.OK, MessageBoxImage.Warning);
84              TypeComboBox.Focus();
85              return;
86          }
87
88          // Заполнение данных
89          _partner.Name = NameTextBox.Text.Trim();
90          _partner.TypeId = (int)TypeComboBox.SelectedValue;
91          _partner.Address = AddressTextBox.Text?.Trim();
92
93          if (!string.IsNullOrWhiteSpace(RatingTextBox.Text))
94          {
95              if (int.TryParse(RatingTextBox.Text, out int rating) && rating >= 0)
96              {
97                  _partner.Rating = rating;
98              }
99              else
100             {
101                 MessageBox.Show("Рейтинг должен быть целым неотрицательным числом",
102                                 "Предупреждение", MessageBoxButton.OK, MessageBoxImage.Warning);
103                 RatingTextBox.Focus();
104                 return;
105             }
106         }
107
108         _partner.DirectorFullname = string.IsNullOrWhiteSpace(DirectorTextBox.Text) ? null : DirectorTextBox.Text.Trim();
109         _partner.Phone = string.IsNullOrWhiteSpace(PhoneTextBox.Text) ? null : PhoneTextBox.Text.Trim();
110         _partner.Email = string.IsNullOrWhiteSpace(EmailTextBox.Text) ? null : EmailTextBox.Text.Trim();
111
112         if (_partner.Id == 0)
113         {
114             _partner.CreatedAt = DateTime.Now;
115             _context.Partners.Add(_partner);
116         }
117         else
118         {
119             _partner.UpdatedAt = DateTime.Now;
120             _context.Entry(_partner).State = EntityState.Modified;
121         }
122
123         _context.SaveChanges();
124         DialogResult = true;
125         Close();
126     }
127     catch (Exception ex)
128     {
129         MessageBox.Show($"Ошибка сохранения: {ex.Message}",
130                         "Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
131     }
73 %    Проблемы не найдены.
```

Рисунок 5.7 – Валидация и сохранение данных

Итоговый вид главного окна приложения представлен на рисунке 5.8.

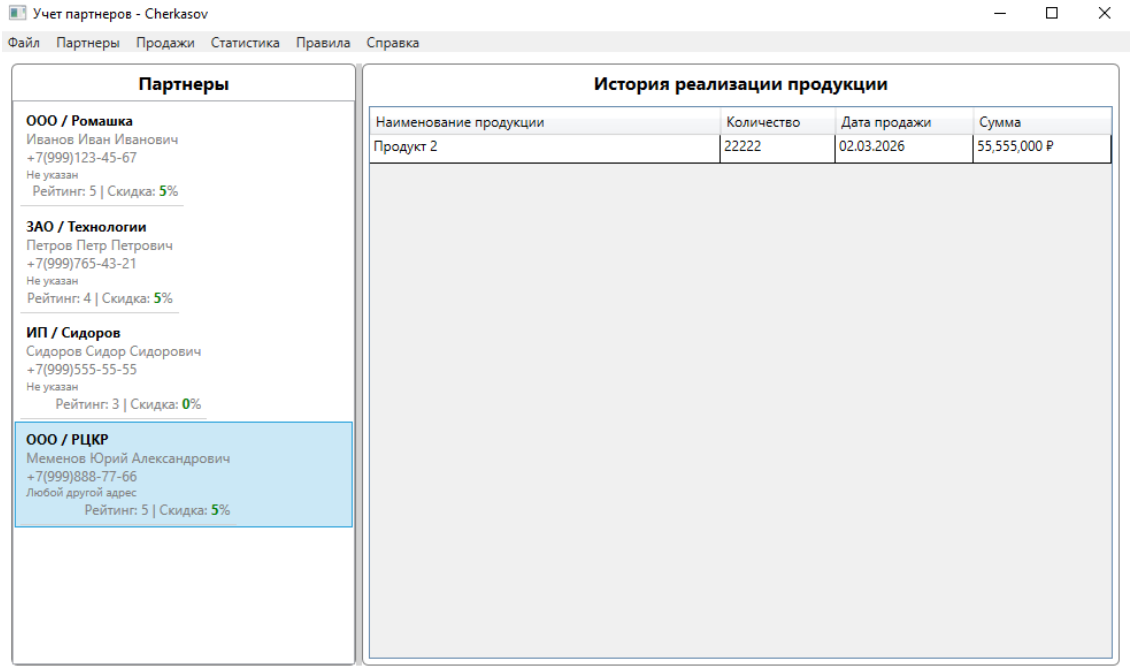

Рисунок 5.8 – Главное окно приложения

Заключение

В результате выполнения индивидуального задания была разработана подсистема для автоматизации учета партнеров компании, обеспечивающая следующий функционал:

- просмотр списка партнеров с отображением актуальной информации о каждом партнере, включая автоматически рассчитанную скидку;
- добавление, редактирование и удаление записей о партнерах с валидацией вводимых данных;
- просмотр истории реализации продукции по каждому партнеру с отображением наименования продукции, количества, даты и суммы продажи;
- добавление, редактирование и удаление записей о продажах с автоматическим расчетом общей суммы;
- автоматический пересчет скидки партнера при изменении данных о продажах в соответствии с заданной шкалой;
- экспорт данных в текстовые файлы;
- обработка исключительных ситуаций с выводом информативных сообщений пользователю.

В ходе выполнения работы были решены следующие задачи:

1. Проведен анализ предметной области и сформулированы функциональные требования к системе.
2. Спроектирована и реализована база данных в СУБД PostgreSQL, приведенная к третьей нормальной форме с обеспечением ссылочной целостности.
3. Созданы триггеры базы данных для автоматического пересчета скидки при изменении продаж.
4. Разработана библиотека классов, содержащая модели данных и контекст для работы с Entity Framework Core.

5. Реализованы ViewModel для подготовки данных к отображению в пользовательском интерфейсе.

6. Созданы модульные тесты с использованием xUnit для проверки ключевых компонентов системы.

7. Разработан пользовательский интерфейс на технологии WPF с главным меню, контекстным меню и диалоговыми окнами.

8. Выполнена интеграция всех компонентов в единое приложение и проведено тестирование.

Разработанное программное обеспечение полностью соответствует требованиям технического задания, успешно проходит тестирование и готово к практическому использованию для автоматизации учета партнеров компании. Приложение имеет интуитивно понятный интерфейс, обеспечивает надежное хранение данных и предоставляет пользователю все необходимые инструменты для эффективной работы.

Приложения

Приложение 1

Описание предметной области

Производственная компания «Мастер пол» выпускает напольные покрытия. Компания занимается производством и реализует свою продукцию через партнеров, которые доставляют продукцию компании до конечных потребителей.

С целью оптимизации деятельности компании разрабатывается система, в которой заказчик выделил несколько подсистем:

- продукция;
- склад и материалы;
- производство;
- сотрудники;
- партнеры.

В данной области определены следующие основные составляющие:

1. Партнеры: тип, наименование компании, юридический адрес, ИНН, ФИО директора, контактные данные (телефон, email), логотип, рейтинг, места продаж, история реализации продукции для последующего расчета скидок.

Партнеры покупают продукцию со скидками, которые зависят от общего объема их продаж. Скидки мотивируют партнеров к большему объему продаж. Партнеры реализуют товары в розничных и оптовых магазинах, интернет-магазинах, другим компаниям по продаже товаров и оказанию услуг.

Взаиморасчеты производственной компании с партнерами осуществляются в рублях.

2. Менеджеры: поиск и регистрация партнеров, решение об изменении рейтинга партнера, ведение истории изменений рейтинга, прием заявок от партнеров, формирование для них предложения на основании статистики истории продаж, полученной от аналитиков компании.

3. Заявка: если партнер ранее производил закупку продукции и предоставил данные о ее реализации, то на основании этих данных и текущих объемах готовой продукции менеджер формирует предложение. Если работа ведется с новым партнером, то менеджер предоставляет каталог готовой продукции и принимает заявку. Затем менеджер должен указать стоимость и дату производства каждой указанной в заявке единицы продукции. После согласования списка продукции с количеством, стоимостью и сроком изготовления заявка считается созданной.

Партнер может отменить заявку до момента внесения предоплаты. Если предоплата не поступает в течение трех дней, то заявка отменяется автоматически, о чем обязательно уведомляется партнер. После поступления предоплаты продукция, указанная в заявке и отсутствующая на складе в нужном количестве, запускается в производство. По готовности всей продукции менеджер должен предложить организовать доставку. В момент получения продукции (путем доставки или самовывоза) партнер производит полную оплату. После проверки количества и качества продукции партнером заявка считается выполненной.

4. Сотрудники: ФИО, дата рождения, паспортные данные, банковские реквизиты, наличие семьи, состояние здоровья.

5. Кадры: допуск к работе с конкретным оборудованием. Мастер производства занимается технологией производства, контролем качества готовой продукции, заказом материалов у поставщиков.

6. Доступ: двери открываются при использовании карты сотрудника, система турникетов передает данные о перемещении сотрудников в приложение для формирования аналитиками статистики по сотрудникам.

7. Материалы (сырье): тип, наименование, поставщик, количество в упаковке, единица измерения, описание, изображение, стоимость, количество на складе, минимальное допустимое количество, история изменений количества

материалов на складе.

8. Склад: регистрация поступлений материалов, резервирование материалов для производства, отпуск и списание материалов со склада в производство, поступление и выбытие готовой продукции, получение информации о текущих остатках и движении товарно-материальных ценностей по складу.

9. Поставщики: тип, наименование, ИНН, история поставок материалов. Менеджеры контролируют список поставщиков, аналитики формируют информацию о качестве поставляемых материалов для принятия решения о прекращении работы с определенными компаниями.

10. Продукция: артикул, тип, наименование, описание, изображение, минимальная стоимость для партнера, размер упаковки (длина, ширина, высота), вес без упаковки, вес с упаковкой, сертификат качества (скан документа), номер стандарта, история изменений минимальной стоимости для партнера, время изготовления, себестоимость, номер цеха, количество человек на производстве, необходимые материалы для производства.